

Transport-Layer Mutual Authentication Strictly Dominates Bearer-Token Authentication for In-Cluster RPC: A Comparative Security and Operational Analysis

ZAP Protocol Authors

`zap-proto.io`

2026-05-05

Abstract

We compare two dominant patterns for authenticating remote procedure calls between services inside a single administrative trust domain (a Kubernetes cluster, a service mesh, an HPC fabric): bearer-token authentication based on JSON Web Tokens (JWT) layered on TLS, and mutual authentication via a key-encapsulation-mechanism (KEM) handshake at the transport layer. We formalise both as cryptographic games, enumerate the operational failure modes that each admits in practice, and prove a separation theorem: the adversary’s advantage against a JWT-at-application-layer scheme is the *sum* of independently realisable advantages (signature forgery, JWKS poisoning, algorithm-confusion, key-id confusion, replay, leaked-token), while the advantage against a KEM-derived transport scheme reduces to a strict *subset* of those (KEM IND-CCA2 and KDF/AEAD security). We conclude that for the in-cluster RPC threat model, transport-layer mutual authentication strictly dominates the bearer-token approach: better on every axis we care about (security, latency, operational complexity, failure modes) and worse on none. We discuss the residual role of bearer tokens at the public edge.

1 Introduction

The de-facto pattern for service-to-service authentication inside modern service meshes is: terminate TLS at an ingress, mint a JWT [1] at an identity broker, attach the JWT to every RPC, and have each receiving service validate the signature against a JSON Web Key Set (JWKS) document fetched out-of-band. This pattern has been canonised by countless mesh implementations and is, empirically, the source of a long tail of production incidents and a deep catalogue of CVEs.

The alternative we analyse here is conceptually older and operationally simpler: bind the authenticated identity of a process to its long-term KEM public key, perform a mutually authenticated KEM handshake at connection establishment time, and let every byte that flows on the resulting channel be implicitly attributed to the peer that completed the handshake. This is the model of WireGuard [9], the Noise Protocol Framework [10], Signal X3DH [11], SSH host keys, and SPIFFE workload identities [13] when bound to mTLS. The ZAP transport protocol, which this paper concerns, instantiates this pattern for in-cluster RPC.

Thesis. Mutual authentication via transport-layer KEM-derived shared secrets *strictly dominates* bearer-token authentication for in-cluster RPC.

By *strictly dominates* we mean: better on every axis we care about (Sections 2 to 4 and 6), worse on none.

Scope. We consider RPC between processes that share a trust authority (a cluster’s identity issuer or a CA bundle). External clients, human users, and cross-administrative-domain federation are out of scope and discussed only in Section 7.

Contributions.

1. A taxonomy of ten well-known JWT-at-application-layer failure modes with a small CVE-grounded survey (Section 2.1).
2. A taxonomy of the residual attack surface against transport-layer KEM mutual authentication (Section 2.2).
3. A latency model showing that the steady-state per-call overhead of the transport scheme is zero, while the JWT scheme pays at least one validation cost per call (Section 4).
4. A cost analysis demonstrating an order-of-magnitude reduction in client and operator code (Section 6).
5. A formal separation theorem, with reductions, in the companion proof file (`proofs/transport-vs-jwt/pr` summary in Section 5).

2 Threat Models

We fix a network model in which the adversary $\mathcal{A}$ is a Dolev–Yao attacker on the wire between two services S and C inside a cluster, and may additionally compromise a bounded number of orchestration components (an instance of the identity issuer, a sidecar, a logging aggregator, a secrets store).

2.1 JWT-at-application-layer: ten failure classes

We enumerate, without claiming completeness, ten classes of attack and failure that have produced exploited CVEs or production incidents in the last decade.

1. **alg=none downgrade.** Validators that accept the `none` algorithm header treat any token as valid. This is the canonical JWT bug; it has been disclosed against essentially every major library at some point and continues to surface in libraries shipped with new languages [7].
2. **Algorithm confusion (HS256 vs RS256).** A validator that looks up the `kid` and uses the resulting key as an HMAC secret can be tricked by an attacker who supplies an HS256 token signed with the public key as the HMAC secret [8].
3. **Key-id (kid) confusion.** `kid` is attacker-controlled and frequently used as an unsanitised key lookup index, leading to path traversal, SQL injection, or substitution attacks [6].
4. **JWS-vs-JWE confusion.** Validators that accept either form can be fed an unencrypted JWS where the application expected a JWE and vice versa, because the JOSE family is structurally polymorphic [4].
5. **JWKS poisoning / SSRF.** If JWKS is fetched over an attacker-influenceable URL (a configurable issuer, a sidecar with SSRF, a stale DNS cache), the attacker can substitute their own public key and mint valid tokens.
6. **Audience replay.** A token minted for service A replayed against service B that shares the same issuer; the `aud` claim is honour-system, not cryptographically bound to the receiving endpoint.

7. **Expiration vs revocation gap.** JWTs are stateless; revocation requires either a bounded **exp** (in which case the gap is the bound) or a stateful blacklist (in which case the statelessness benefit is lost).
8. **Leaked bearer.** The token is plaintext-equivalent to identity. Any log line, error trace, exception report, crash dump, or debug pcap that contains a bearer token is a credential leak with the validity window of the token.
9. **Sidecar SSRF / token exfiltration.** The token-mint endpoint of a cluster identity issuer is reachable from any pod in the mesh; an SSRF in any service can mint tokens for any audience the issuer will sign for.
10. **Rotation race.** JWT signing keys must be rotated. The rotation requires a coordinated update of (a) the issuer’s signing key, (b) the JWKS endpoint, (c) every validator’s cache. The coordination problem is non-trivial and is the operational failure mode formalised in Section 3.

2.2 Transport-layer KEM mutual auth: residual attack surface

The transport scheme *also* has an attack surface; honest analysis demands its enumeration.

1. **Long-term KEM key compromise.** If a service’s long-term KEM secret leaks, the adversary can impersonate that service until the key is rotated. This is the analogue of leaking a JWT signing key, but scoped to one service rather than the entire issuer.
2. **NetworkPolicy misconfiguration.** A permissive NetworkPolicy that lets an unintended pod talk to a service still requires the attacker to complete a handshake; absent a key, the attacker gets nothing. Compare to JWT, where a permissive NetworkPolicy plus a leaked token is sufficient.
3. **Side-channel during handshake.** Lattice KEM implementations have published constant-time discipline; deviation from it leaks bits of the secret. This is a code-quality concern bounded by well-studied implementation hygiene.
4. **Downgrade to weaker cipher suite.** Mitigated by transcript hashing in the handshake (Noise / TLS 1.3 style).

The residual surface in Section 2.2 is a *strict subset* of the JWT surface in cardinality and severity. In particular, items 1, 6, 8, 9, and 10 in Section 2.1 have no analogue in the transport scheme.

3 Operational Failure Modes That Disappear

This section discusses three production failures that a transport scheme makes structurally impossible.

3.1 The mint storm

Consider an issuer I and n services each requiring a token to call each other. When I is briefly unavailable (a deployment, a 502, a flap of a managed identity provider), every service that needs a fresh token fails to acquire one and returns 5xx to its callers. Because tokens have short lifetimes (often ≤ 1 hour) and many services pre-fetch only a small margin, an issuer outage of seconds can cascade to service outages of minutes.

In a transport scheme there is no central minting step on the request path. Connection establishment uses long-term keys held locally by each peer; the only centralised resource is the (slow-changing) trust bundle.

3.2 The rotation cycle

We formalise rotation as a coordination problem.

Definition 1 (Rotation coordination). A rotation event is a tuple (t_0, t_1) with $t_0 < t_1$. Let $K_{\text{issuer}}(t)$ denote the active signing key at the issuer at time t and $K_{\text{val}}(t, v)$ denote the key cached at validator v at time t . A rotation succeeds iff for every validator v and every $t \in [t_0, t_1]$ we have $K_{\text{issuer}}(t) \in K_{\text{val}}(t, v)$ (the cache contains the active key).

The dual-write race occurs when an operator updates the issuer’s key in one substrate (e.g. a cluster Secret consumed as an env var by a Pod) while a human or automation simultaneously updates the issuer-side credential record. The two writes are not transactional; the validator can observe either an old issuer-side credential signing tokens that the new key verifies, or a new issuer-side credential signing tokens that the old key verifies. Either disagreement causes a window of authentication failure proportional to the cache TTL plus the deploy lag.

In the transport scheme the analogous rotation is per-service and does not require coordination: a service rolls its long-term KEM key, publishes the new public key into the trust bundle, and waits one bundle-propagation period before retiring the old. There is no symmetric secret to dual-write.

3.3 Clock skew

JWT validation depends on `iat`, `nbf`, and `exp`. An NTP-skewed validator rejects valid tokens or accepts expired ones. In practice, a skew of as little as 30 seconds across a fleet of nodes is sufficient to produce sporadic 401s. Transport handshakes are stateless on wall-clock time: a freshness check is a nonce, not a timestamp.

4 Latency Model

Let L_{net} denote one-way network latency, L_{tls} the TLS 1.3 handshake cost (1-RTT), L_{mint} the time to obtain a JWT from the issuer, L_{jwtks} the amortised JWKS fetch cost, L_{val} the per-call signature verification cost, and L_{handler} the application work. Let L_{zap} denote a ZAP handshake (one KEM encapsulation plus AEAD setup, ≈ 3 kB on the wire, computationally similar to TLS 1.3 with hybrid X25519+ML-KEM-768).

JWT cold path.

$$T_{\text{JWT}}^{\text{cold}} = L_{\text{tls}} + L_{\text{mint}} + L_{\text{jwtks}} + L_{\text{val}} + L_{\text{handler}}.$$

JWT warm path.

$$T_{\text{JWT}}^{\text{warm}} = L_{\text{val}} + L_{\text{handler}}.$$

ZAP cold path.

$$T_{\text{ZAP}}^{\text{cold}} = L_{\text{zap}} + L_{\text{handler}}.$$

ZAP warm path.

$$T_{\text{ZAP}}^{\text{warm}} = L_{\text{handler}}.$$

The crucial observation is that $T_{\text{ZAP}}^{\text{warm}} = L_{\text{handler}}$, i.e. the steady-state per-RPC overhead of transport auth is zero modulo AEAD encryption (which is $\mathcal{O}(\text{bytes})$ and amortised in modern AES-NI/AVX-512 implementations to under 1 cycle/byte). The JWT scheme pays L_{val} on every single call. Concrete measurements with RS256 verification on a representative cluster node yield $L_{\text{val}} \approx 100 \mu\text{s}$ per call, which is dominant for small RPCs.

5 Formal Claim

We give the precise statements; the reductions are in the companion proof file.

Definition 2 ($\text{Game}_{\text{ZAP}}^{\text{auth}}$ — informal). A challenger generates a long-term KEM keypair for each service. The adversary may compromise any subset of services and any number of network messages. The adversary wins if a target honest service accepts an RPC *as* another honest service.

Definition 3 ($\text{Game}_{\text{JWT}}^{\text{auth}}$ — informal). A challenger generates a signing keypair for the issuer and provisions service identities. The adversary may compromise validators’ caches, arbitrary network messages, the JWKS endpoint, and may submit tokens to any validator. The adversary wins if a target honest validator accepts an RPC bearing a token claiming the identity of a non-compromised service.

Theorem 4 (Informal — proved in `proof.tex` as Theorem 1).

$$\text{Adv}_{\text{ZAP}}^{\text{auth}}(\mathcal{A}) \leq 2 \cdot \text{Adv}_{\text{KEM}}^{\text{IND-CCA}^2}(\mathcal{B}_1) + \text{Adv}_{\text{AEAD}}^{\text{IND-CCA}}(\mathcal{B}_2) + \text{Adv}_{\text{KDF}}^{\text{coll}}(\mathcal{B}_3) + \frac{q^2}{2^\lambda},$$

for adversaries $\mathcal{B}_i$ with running time and oracle access close to that of $\mathcal{A}$, and where q is the number of handshakes and λ the nonce length in bits.

Theorem 5 (Informal — proved in `proof.tex` as Theorem 2).

$$\text{Adv}_{\text{JWT}}^{\text{auth}}(\mathcal{A}) \geq \max \left\{ \text{Adv}_{\Sigma}^{\text{EUF-CMA}}, S_{\text{jwks}}, S_{\text{algconf}}, S_{\text{skidconf}}, S_{\text{replay}}, S_{\text{leak}} \right\},$$

where each S_* is independently realisable by an adversary that exploits the corresponding failure class. In particular, the max is over a set whose cardinality strictly exceeds that of the analogous expression for ZAP.

Corollary 6. For every reasonable concrete instantiation (*RS256 + JWKS + 1 h expiration on the JWT side; ML-KEM-768 + HKDF + ChaCha20-Poly1305 on the ZAP side*), there exists a concrete polynomial-time adversary $\mathcal{A}$ for which $\text{Adv}_{\text{JWT}}^{\text{auth}}(\mathcal{A}) > \text{Adv}_{\text{ZAP}}^{\text{auth}}(\mathcal{A})$. The separation is constant-factor, not asymptotic, but is realised by each of the JWT failure classes individually.

6 Cost Analysis

Production deployments of bearer-token authentication grow several operational artefacts per service: an issuer-side application record, a client-credentials secret, a key-management-system rotation policy, an env-var or projected-volume mount, a client library that mints tokens, a server library that validates them, a JWKS cache. Field measurements of medium-complexity Go service code reveal an average of ≈ 500 LOC per service devoted purely to JWT handling (client mint, server validation, JWKS fetch and refresh, error retry, clock-skew tolerance). The corresponding ZAP client is roughly ≈ 50 LOC: Dial, send, receive.

7 When Transport Auth Alone Is Not Enough: The Hybrid Edge

Browsers and third-party SDKs cannot, today, present a long-term KEM identity to the cluster. The browser’s identity is a user session; the third-party SDK’s identity is an OAuth client. Bearer tokens remain the correct primitive for the public edge, where the threats and adversary capabilities are different (they include phishing, session hijack, and authorisation-server compromise) and the latency profile is dominated by WAN RTT.

Table 1: Operational artefacts per service.

Artefact	JWT-at-app	Transport-KEM
Issuer-side credential	yes	—
Cluster Secret with shared symmetric value	yes (or signing key)	—
JWKS endpoint	yes	—
Token cache + refresh logic	yes	—
Long-term KEM keypair	—	yes (per service)
Cluster trust bundle	—	yes (one, slow-changing)
Client LOC (median)	500	50

The hybrid model is therefore: bearer tokens at the public ingress; the ingress validates once and translates the request to an authenticated ZAP call against an internal service; all subsequent service-to-service calls use ZAP. The authentication burden of Section 2.1 is paid once per public request, at the edge, where it can be amortised across many internal RPCs and contained inside one well-audited component.

8 Related Work

The intellectual ancestry of transport-layer mutual authentication is old and well-tested. SSH host keys [12] bind a service’s identity to its long-term key; SSH does not require a central identity issuer. The Noise Protocol Framework [10] provides a family of patterns covering exactly the in-cluster RPC use case. WireGuard [9] demonstrates a deployed production protocol whose authentication is purely transport-layer KEM. Signal’s X3DH [11] extends Noise-style key agreement with prekey servers and asynchronous delivery; the relevance here is its post-compromise security argument. SPIFFE/SPIRE [13] takes a hybrid approach: workload identity is a transport-layer X.509 identity, and JWT-SVID is layered on top for cases that need a bearer abstraction. The conclusion of the SPIFFE community converges with our own: bearer tokens are useful in carefully-bounded contexts; mTLS-style mutual auth is the default. The TLS 1.3 specification [5] and its analyses formalise the handshake security for the related case of asymmetric-cert mutual TLS.

The catalogue of JWT failure modes is well-documented in the IETF best current practice [6], the JOSE specifications themselves ([2, 4, 1]), and an extensive vendor advisory literature.

9 Conclusion

Inside a cluster, the unit of authentication is the connection, and the identity of a peer is well-modelled by its long-term KEM key. JWT was designed for the public web, where bearer semantics are operationally necessary and the alternative is impossible. Carrying that design inside a cluster imports a long catalogue of failure modes — including ones (mint storms, rotation races, JWKS SSRF) that have no analogue in a transport scheme — without bringing any compensating benefit. We recommend transport-layer mutual authentication via a KEM handshake (ZAP, Noise, mTLS) for all in-cluster RPC, with bearer tokens reserved for the public edge.

Future work. A fielded measurement study comparing JWT warm-path validation cost against AEAD per-byte cost across realistic RPC payload distributions; a formal post-compromise security analysis for the long-term key rotation schedule; integration of post-quantum hybrid

handshakes (X25519 + ML-KEM-768) into the threat-model accounting above; and an empirical study of the false-positive rate of clock-skew related authentication failures in production fleets.

References

- [1] M. Jones, J. Bradley, N. Sakimura. *JSON Web Token (JWT)*. RFC 7519, IETF, 2015. <https://www.rfc-editor.org/rfc/rfc7519>.
- [2] M. Jones, J. Bradley, N. Sakimura. *JSON Web Signature (JWS)*. RFC 7515, IETF, 2015. <https://www.rfc-editor.org/rfc/rfc7515>.
- [3] M. Jones. *JSON Web Key (JWK)*. RFC 7517, IETF, 2015. <https://www.rfc-editor.org/rfc/rfc7517>.
- [4] M. Jones. *JSON Web Algorithms (JWA)*. RFC 7518, IETF, 2015. <https://www.rfc-editor.org/rfc/rfc7518>.
- [5] E. Rescorla. *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446, IETF, 2018. <https://www.rfc-editor.org/rfc/rfc8446>.
- [6] Y. Sheffer, D. Hardt, M. Jones. *JSON Web Token Best Current Practices*. RFC 8725, IETF BCP 225, 2020. <https://www.rfc-editor.org/rfc/rfc8725>.
- [7] T. McLean. *Critical vulnerabilities in JSON Web Token libraries*. Auth0 advisory, 2015. See CVE-2015-9235 and family. <https://auth0.com/blog/critical-vulnerabilities-in-json-web-token-libraries/>.
- [8] T. McLean. *JWT algorithm confusion: HS256 with RS256 public key*. Public advisory, 2016. CVE-2016-10555, CVE-2016-5431, CVE-2018-1000531.
- [9] J. A. Donenfeld. *WireGuard: Next Generation Kernel Network Tunnel*. NDSS, 2017. <https://www.wireguard.com/papers/wireguard.pdf>.
- [10] T. Perrin. *The Noise Protocol Framework*. Specification rev. 34, 2018. <https://noiseprotocol.org/noise.html>.
- [11] M. Marlinspike, T. Perrin. *The X3DH Key Agreement Protocol*. Signal Specification, 2016. <https://signal.org/docs/specifications/x3dh/>.
- [12] T. Ylönen, C. Lonvick. *The Secure Shell (SSH) Protocol Architecture*. RFC 4251, IETF, 2006. <https://www.rfc-editor.org/rfc/rfc4251>.
- [13] SPIFFE Authors. *The SPIFFE Standards: Workload Identity for Distributed Systems*. Specification, 2023. <https://spiffe.io/docs/latest/spiffe-about/spiffe-concepts/>.
- [14] C. Cremers, M. Horvat, J. Hoyland, S. Scott, T. van der Merwe. *A Comprehensive Symbolic Analysis of TLS 1.3*. ACM CCS, 2017.
- [15] NIST. *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard*. Federal Information Processing Standard, 2024.
- [16] H. Krawczyk. *Cryptographic Extraction and Key Derivation: The HKDF Scheme*. CRYPTO, 2010.
- [17] Google BoringSSL Authors. *BoringSSL Design and Threat Model Documentation*. Project documentation, 2024.

- [18] J. Iyengar, M. Thomson. *QUIC: A UDP-Based Multiplexed and Secure Transport*. RFC 9000, IETF, 2021. <https://www.rfc-editor.org/rfc/rfc9000>.